Semantic XPath 结构化记忆访问方法阅读报告

oioioi-521

(天津大学 NLP 第 8 队)

摘　要:Semantic XPath 结合路径、位置条件与语义评分访问树形对话记忆。本文介绍其数据模型、查询执行与证据聚合，分析单轮、多轮和 LoCoMo 实验，并讨论准确性与输入开销的取舍。该方法为结构化访问提供了可解释路径，其效果取决于层级质量、评分器和任务需求。

关键词: 结构化记忆；对话智能体；Semantic XPath；语义检索；论文阅读

中图法分类号: TP18

A Reading Report on Semantic XPath for Structured Conversational Memory Access

oioioi-521

(NLP Team 8, Tianjin University)

1　Semantic XPath

Semantic XPath[1]研究结构化对话记忆的访问。它结合路径、位置条件和连续语义评分，从已有树形记忆中选择相关节点及子树，支持下游回答和状态更新。

1.1　研究问题与方法定位

长期任务导向对话需要反复查看、修改和回溯同一个任务对象。旅行计划中的一项活动不仅包含文本，还属于特定日期、行程与历史版本。全量上下文保留信息较多，但输入量随历史增长；扁平检索降低输入量，却可能取回语义相似而所属范围错误的条目。比如 “在会议安排最密集的一天增加咖啡休息”，需要比较各天的活动，而不仅是找一条包含会议内容的文本。

查询负责定位访问范围，插入、删除及版本维护由下游步骤完成。

1.2　树形数据模型与查询语法

记忆表示为有根树 $T = (V, E, r)$。非根节点只有一个父节点，节点具有类型和文本属性；模式 Σ 由记忆数据已有的节点类型、属性和父子关系决定，而非要求人工预先设计一个通用模式。旅行示例采用

$$\text{Itinerary} \rightarrow \text{Version} \rightarrow \text{Day} \rightarrow \text{POI},$$

分别表示计划、版本、日期和活动；层级主要是组成关系，不只是对下层文本做有损摘要。

查询是步骤序列，每一步写为 $S = A\,N\,[R]\,[P]$：路径与类型必需，位置选择器和语义条件可选。

表 1　Semantic XPath 查询步骤的组成

组成	示例写法	作用
路径 A	`/`、`//`	访问直接子节点或全部后代
类型 N	`Day`、`POI`、`*`	按节点类型筛选或匹配任意类型
位置 R	`[i]`、`[-i]`、`[i:j]`	按顺序、倒序或区间选取候选
语义 P	`[node~="workshop"]`	计算候选与语义条件的相关程度

例如，`//Day[3]/POI[node~="workshop"]` 先选第三个匹配日期，再对该日期下的活动评分。存在多个行程

或版本时，查询需先限定目标范围。语义条件 $1-P$ 对原条件评分取反，提高低分候选的优先级。

1.3 语义评分与带权执行

语义相关度定义为 $\mathrm{Rel}(u,P)\in[0,1]$。局部评分依据一个属性或完整节点表示；聚合评分先在当前节点下执行内部结构查询，再用平均、最小、最大或几何平均聚合证据；多个条件还可用乘积、平均及取反组合。

```
//Day[avg(/POI[node~="conference"])]
```

这个查询根据活动证据评价日期。示例中第二天的三个活动分数为 0.603、0.482、0.608，平均约 0.565，在候选日期中得分最高。该值衡量活动与会议主题的平均相关度。

执行器从根节点及权重 1 出发，维护带权集合 $W\subseteq V\times[0,1]$，按步骤进行路径扩展、类型筛选、位置选择与语义评价。语义步骤更新为

$$w_{\mathrm{new}}=w_{\mathrm{old}}\,\mathrm{Rel}(u,P). \tag{1}$$

聚合条件递归访问下层证据。阈值、返回数量和空集处理由具体任务设置，查询质量则可从目标范围、节点选择和证据完整性三个方面检查。

1.4 范围约束示例

以行程调整为例，假设当前版本的第 2 天安排海滨公园，第 5 天安排山地徒步，第 7 天安排植物园和海港游船。用户要求找出“第 7 天下雨时需要调整的户外活动”。纯语义排序可能混入其他日期的活动；结构化访问先限定第 7 天，再评价活动语义，目标为植物园与游船。

简单日期约束也可由带元数据过滤的扁平 RAG 完成。比较查询机制时，可将其作为基线，进一步考察嵌套范围、下层证据聚合等结构访问需求。

1.5 实验设置与指标口径

作者评估旅行行程、待办清单和家庭配餐三个自建场景，每场景 20 个单轮对话、5 个多轮对话；另在 LoCoMo 中使用原生的 Memory–Conversation–Session–Turn 层级。回答模型为 GPT-5 mini 与 Gemini 3 Flash。相似度评分器（Sem）和 Flat RAG 使用 Qwen3-Embedding-8B；蕴含评分器（Entail）使用 `facebook/bart-large-mnli`。

自建场景使用任务通过率，要求检索到正确结构化数据且输出满足任务约束；LoCoMo 使用模型裁判评定的回答得分。LoCoMo 的平均输入 tokens 包含 XPath 查询生成输入，自建场景则报告平均 token 用量。成本比较还需计入嵌入、蕴含评分和查询执行，并在同一模型内对齐分词与统计口径。

表 2 三个自建场景的平均单轮结果

回答模型	方法	通过率（%）	平均 tokens
GPT-5 mini	In-context	70.0	26 216
	Flat RAG	30.0	3 250
	XPath Sem	66.3	6 166
	XPath Entail	83.0	5 691
Gemini 3 Flash	In-context	79.0	25 387
	Flat RAG	37.7	3 361
	XPath Sem	64.3	6 043
	XPath Entail	70.7	5 830

单轮 GPT-5 mini 配置中，Entail 相对 Flat RAG 提高 53.0 个百分点，相对增幅约 176.7%；token 用量约为全量上下文的 21.7%。Gemini 配置中，全量上下文的 79.0% 高于 Entail 的 70.7%，评分机制的收益随模型变化。

表 3 三个自建场景的平均多轮结果

回答模型	方法	通过率（%）	平均 tokens
GPT-5 mini	In-context	80.0	69 097
	Flat RAG	26.7	3 327
	XPath Sem	70.0	6 294
	XPath Entail	83.3	5 850
Gemini 3 Flash	In-context	56.7	76 603
	Flat RAG	26.7	3 240
	XPath Sem	76.7	6 092
	XPath Entail	80.0	5 993

多轮 Entail 在两种模型下取得最高平均通过率，token 用量约为全量上下文的 8.5% 与 7.8%。Sem 与 Entail 的差异说明评分器选择会影响任务效果。单轮与多轮实验使用不同请求，分别反映两类任务设置。

表 4 LoCoMo 结果（XPath 输入含查询生成）

回答模型	方法	回答得分	平均输入 tokens
GPT-5 mini	In-context	74.55	24 857
	Flat RAG	35.71	1 341
	Semantic XPath	65.19	3 032
Gemini 3 Flash	In-context	87.01	27 499
	Flat RAG	41.49	1 996
	Semantic XPath	73.25	3 870

LoCoMo 上 XPath 的总体得分处于两种基线之间，输入分别约为全量上下文的 12.2% 与 14.1%。这说明已有会话层级可用于访问，但准确性仍有损失。例如 GPT-5 mini 多跳题得分为 43.26，低于全量上下文 58.87，证据整合仍有差距。作者的失败分析将错误分为查询生成、检索和回答生成，认为检索是主要剩余来源，包括无命中、证据不全及选错节点或发言。

1.6 方法局限与课设启发

结构范围与连续语义评分支持不同粒度的访问，路径与评分轨迹便于解释结果。方法依赖可靠层级，跨树关系、模糊版本和复杂多跳仍有困难。现有评测规模较小，重复实验、置信区间及本地评分开销也有待补充。

StructMem 侧重写入时保存事件关联，Semantic XPath 侧重读取时限定范围。课程设计可明确日期与版本约束，保留节点及来源标识，分别检查范围、证据完整性和回答错误，并以元数据过滤基线评估层级访问的贡献。

References:

[1] Liu Y S, Wu R, Gallagher L, et al. Semantic XPath: Structured agentic memory access for conversational AI. In: Proceedings of Durrett G, Jian P, editors, Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), San Diego, California, United States: Association for Computational Linguistics, 2026. 286–296